

MPC5748G Automotive Cluster & Gateway Training Project

Detailed Requirement Specification

Field	Description
Document type	Technical requirement specification
Target platform	6 x MPC5748G boards, CANoe VN5650, Trace32, 3 x Arduino sensor emulator boards
Training goal	Enable mentees to understand, implement, debug, test, and integrate automotive embedded software on MPC5748G.
System concept	Six ECU nodes connected through CAN, monitored and tested by CANoe, debugged with Trace32, with Arduino-based sensor emulation.
Version	1.0.0
Date	2026-05-18

Document Control

Version	Date	Author/Owner	Change Summary
v1.0.0	2026-05-19	Mentor group	Initial version

Table of Contents

Contents

Document Control	2
Table of Contents	3
1. Project Overview and Training Scope.....	4
2. System Architecture.....	4
3. Requirement Conventions.....	5
4. Common Requirements for All Mentees.....	5
4.1 Common Software Structure	6
5. ECU1 - Body Control Node Requirements.....	6
Required deliverable folder.....	7
6. ECU2 - Powertrain Node Requirements.....	8
Required deliverable folder.....	9
7. ECU3 - Sensor Fusion Node Requirements.....	9
Required deliverable folder.....	11
8. ECU4 - Cluster Display Node Requirements	11
Required deliverable folder.....	12
9. ECU5 - Diagnostic and UDS Node Requirements	13
Required deliverable folder.....	14
10. ECU6 - Gateway, OTA, and Security Node Requirements.....	15
Required deliverable folder.....	17
11. Integration Scenarios.....	17
12. CAN Matrix.....	18
13. CANoe, Trace32, and Arduino Expectations.....	19
13.1 CANoe expectations.....	19
13.2 Trace32 expectations	19
13.3 Arduino expectations	19
14. Required Deliverables and Folder Structure.....	19
15. Evaluation Rubric and Review Checklist	20
15.1 Mentor review checklist	20
Appendix A - Suggested UDS DID and DTC Table	21
Appendix B - Recommended Final Demo Flow	21

1. Project Overview and Training Scope

This document defines the detailed technical requirements for a six-mentee automotive embedded training project using MPC5748G boards. Each mentee owns one ECU role. The six ECUs must communicate over CAN and integrate into a simulated vehicle cluster and gateway system.

The project is intentionally designed so that the ECU roles are different. This prevents direct copying while still forcing system-level integration. The expected implementation effort is approximately one focused week for coding and debugging, followed by integration, CANoe testing, Trace32 review, and final demonstration.

- Mentees must understand hardware initialization, UART logging, GPIO, timers, CAN communication, error handling, and state-machine design.
- Mentees must use CANoe VN5650 for CAN trace analysis, DBC-based signal decoding, panel creation, CAPL tests, and fault injection.
- Mentees must use Trace32 to debug the target, load code, set breakpoints, inspect variables/registers, and create reusable CMM scripts.
- Arduino boards are used as external sensor emulators through I2C/SPI and may also provide UART-based log viewing or input injection.
- Advanced roles include UDS diagnostics, DTC handling, gateway filtering, OTA update simulation, and software-based security/HSM mock behavior.

ECU	Owner	Role	Difficulty	Main Skills
ECU1	Cuong (PM) * Master schedule & Detail plan (each member) * Prepare kick-off meeting between customer and team.	Body Control Node	1.5/5	GPIO, CAN Tx/Rx, door/seatbelt logic, turn/hazard signals, CANoe panel
ECU2	Minh Thu (DEV)	Powertrain Node	2.5/5	Sensor input, RPM/speed processing, overheat logic, limp mode, fault handling
ECU3	ChangHwan (DEV)	Sensor Fusion Node	3/5	I2C/SPI, filtering, range validation, timeout and stuck-sensor detection
ECU4	YouJin (DEV)	Cluster Display Node	3/5	Multiple CAN Rx messages, warning lamps, state machine, fail-safe display
ECU5	YoungSeo (DEV)	Diagnostic/UDS Node	4/5	UDS services, sessions, DID, DTC, negative responses, security access
ECU6	JunSang (Team Leader)	Gateway + OTA + Security Node	5/5	Gateway monitoring/filtering, checksum/alive validation, OTA protocol, security mock

Final outcome:

2. System Architecture

The system consists of six MPC5748G boards connected to the same CAN network. CANoe/VN5650 acts as the simulation and test environment. Trace32 is used for target debug and automation. Three Arduino boards provide simulated sensor data and optional UART monitoring.

CANoe VN5650 <-> CAN Bus <-> ECU1 / ECU2 / ECU3 / ECU4 / ECU5 / ECU6

Trace32 <-> each MPC5748G board for debug, flashing, breakpoints, and CMM automation

Arduino 1 <-> ECU1/ECU4: body sensor emulator

Arduino 2 <-> ECU2/ECU6: powertrain sensor emulator and optional debug input

Arduino 3 <-> ECU3/ECU5: environment sensor emulator and optional diagnostic trigger

Component	Role	Minimum Expectation
MPC5748G board x6	Main embedded ECU nodes	Boot, UART log, CAN Tx/Rx, application logic, Trace32 debug support
CANoe VN5650	CAN simulation and testing	DBC decode, panels, CAPL test cases, trace logging, fault injection
Trace32	Debug and automation tool	CMM scripts, reset/load/run, breakpoints, watch variables, memory/register inspection
Arduino x3	Sensor emulator boards	I2C/SPI sensor data, UART log bridge, sensor fault injection

3. Requirement Conventions

Keyword	Meaning
MUST	Mandatory. The mentee cannot pass the project review without satisfying this requirement.
SHOULD	Strongly recommended. Missing this may reduce score but may not block integration.
MAY	Optional enhancement or bonus feature.
P0	Critical for system integration and final demonstration.
P1	Important for individual evaluation and maintainability.
P2	Advanced or bonus requirement.

Each requirement includes an ID, priority, requirement statement, acceptance criteria, and required evidence. Evidence can be source code, CANoe trace, CAPL result, UART log, Trace32 screenshot, CMM script, design document, or live demonstration.

4. Common Requirements for All Mentees

Requirement ID	Priority	Requirement Statement	Acceptance Criteria	Evidence / Deliverable
REQ-COM-001	P0	Each ECU MUST initialize system clock, GPIO, UART, CAN controller, timer/scheduler, and required interrupts.	The board boots successfully, prints a boot log, toggles a heartbeat LED, and runs for at least 10 minutes without abnormal reset.	Initialization source code, UART log, live demo.
REQ-COM-002	P0	Each ECU MUST provide UART logging with a consistent format: [time][ECUx][module][level] message.	Boot, init, CAN_TX, CAN_RX, application state, warning, and error events are visible. UART logging must not flood the system.	UART terminal screenshot or saved log file.
REQ-COM-003	P0	Each ECU MUST transmit at least one periodic CAN message and receive at least one CAN message from another ECU or CANoe.	CANoe trace shows the message, DBC decodes all mandatory signals, and receive processing is observable.	CANoe trace screenshot, DBC file, source code.
REQ-COM-004	P0	Each ECU MUST implement a 10 ms, 100 ms, and 1000 ms task or an equivalent scheduler.	Tasks execute periodically; no long blocking delay is used in the main control loop.	Scheduler code, Trace32 breakpoint evidence.
REQ-COM-005	P0	Each ECU MUST implement timeout monitoring for critical received CAN messages or external sensor inputs.	When input stops beyond the timeout threshold, the ECU sets an error flag, sends error status on CAN, and logs the event.	Fault injection test, CANoe trace.
REQ-COM-006	P0	Each ECU primary status message MUST include a 4-bit alive counter.	Alive counter increments from 0 to 15 and wraps to 0. Gateway/CANoe can detect if it stops changing.	DBC signal and CANoe trace/graph.
REQ-COM-007	P0	Each ECU primary status message MUST include a simple checksum.	Checksum is calculated consistently; CANoe or ECU6 can detect a deliberately corrupted checksum.	Checksum function, CAPL test.
REQ-COM-008	P1	Each ECU SHOULD separate driver, service, and application layers.	Main.c is not overloaded with all logic; modules are clearly separated.	Source tree review.
REQ-COM-009	P1	Each ECU SHOULD implement a state machine when application behavior has multiple modes.	States and transitions are clear, logged, and testable.	State machine design document and code.
REQ-COM-010	P0	Each mentee MUST create init_debug.cmm and test_debug.cmm for Trace32.	Mentor can connect, reset, load ELF, set breakpoints, and inspect variables using the scripts.	CMM files and Trace32 screenshot.
REQ-COM-011	P0	Each mentee MUST create at least three CAPL test cases for the assigned ECU.	At least one normal case, one invalid input case, and one timeout/fault case must produce PASS/FAIL result.	CAPL file and CANoe test result.
REQ-COM-012	P0	Each mentee MUST provide design and test documentation.	The documentation explains role, inputs, outputs, CAN messages, logic, error handling, and test evidence.	design.md and test_report.md.
REQ-COM-013	P1	Each ECU SHOULD support fault recovery when inputs or messages return to valid condition.	Error flags clear according to defined recovery conditions, and the ECU returns to normal operation.	Recovery test evidence.

Requirement ID	Priority	Requirement Statement	Acceptance Criteria	Evidence / Deliverable
REQ-COM-014	P1	Each ECU SHOULD use traceable names and comments linking important logic to requirement IDs.	Mentor can map code and tests back to requirements.	Code review.
REQ-COM-015	P2	Each ECU MAY include host-side unit tests for pure logic such as checksum, filters, warning rules, and state transitions.	Unit tests can be executed without hardware.	Unit test source and execution log.

4.1 Common Software Structure

```

project_root/
  drivers/
    can_driver.c
    uart_driver.c
    gpio_driver.c
    timer_driver.c
    spi_driver.c or i2c_driver.c
  services/
    logger.c
    checksum.c
    timeout_monitor.c
    uds.c / ota.c / security.c when applicable
  app/
    app_main.c
    app_state_machine.c
    app_signal_process.c
  cmm/
    init_debug.cmm
    test_debug.cmm

```

5. ECU1 - Body Control Node Requirements

ECU1 simulates body-control functions including door status, seatbelt status, turn signal, hazard signal, and body warning output. It is the easiest ECU and focuses on GPIO, periodic CAN transmission, simple application logic, and CANoe panel interaction.

Input requirements

Input	Source	Range / Encoding	Notes
Door_FL, Door_FR, Door_RL, Door_RR	Arduino or CANoe	0 = closed, 1 = open	Invalid values must be rejected and logged.
Seatbelt	Arduino or CANoe	0 = not fastened, 1 = fastened	Used for seatbelt warning logic.
TurnSignalMode	CANoe panel or Arduino switch	0 = off, 1 = left, 2 = right, 3 = hazard	Controls LED blinking and CAN status.
VehicleSpeed	ECU2 or CANoe	0..220 km/h	Used for door/seatbelt warning logic.

Output requirements

Output	Message / Interface	Cycle / Trigger	Content
0x101	BodyStatus	100 ms	Door bits, Seatbelt, TurnSignalMode, HazardActive, BodyWarning, BodyInputError, AliveCounter, Checksum
GPIO LEDs	Body indicators	Event/periodic	Door warning, seatbelt warning, left/right blink, hazard, error indicator
UART	Debug log	Event/periodic	Boot, input update, CAN Tx/Rx, warning activation, errors

Detailed requirements

Requirement ID	Priority	Requirement Statement	Acceptance Criteria	Evidence / Deliverable
REQ-ECU1-001	P0	ECU1 MUST read or receive four door states and validate each value.	CANoe/Arduino can set each door open/closed; invalid values set BodyInputError.	CANoe panel, UART log, test case.
REQ-ECU1-002	P0	ECU1 MUST read or receive seatbelt status and include it in BodyStatus.	Seatbelt state changes are reflected in CANoe within one BodyStatus cycle.	CANoe trace.
REQ-ECU1-003	P0	ECU1 MUST support turn signal modes: off, left, right, and hazard.	Left/right LEDs blink at approximately 500 ms; hazard blinks both sides.	Live demo and UART log.
REQ-ECU1-004	P0	ECU1 MUST send BodyStatus on CAN ID 0x101 every 100 ms.	CANoe measures stable 100 ms cycle and decodes all signals.	CANoe trace and DBC.
REQ-ECU1-005	P0	ECU1 MUST set BodyWarning if vehicle speed is greater than 5 km/h and any door is open.	Speed 40 km/h + Door_FL open results in BodyWarning = 1. Speed 0 may remain normal.	CAPL test TC_ECU1_DoorOpenDriving.
REQ-ECU1-006	P0	ECU1 MUST set BodyWarning if speed is greater than 10 km/h and seatbelt is not fastened.	Speed 20 km/h + Seatbelt = 0 results in BodyWarning = 1.	CAPL test TC_ECU1_SeatbeltWarning.
REQ-ECU1-007	P1	ECU1 SHOULD debounce physical/simulated switch inputs.	Short glitches do not cause unstable CAN signals.	Debounce test or code review.
REQ-ECU1-008	P0	ECU1 MUST detect input timeout if the selected input source is not updated for more than 500 ms.	Input timeout sets BodyInputError and logs the error.	Timeout test.
REQ-ECU1-009	P1	ECU1 SHOULD recover from input timeout when valid input resumes.	BodyInputError clears after three consecutive valid samples.	Recovery test.
REQ-ECU1-010	P1	ECU1 SHOULD provide a Body Control CANoe panel.	Panel can control door, seatbelt, turn mode and display warning/error flags.	Panel screenshot.

Required CANoe panel elements

CANoe panel element	Purpose
Door_FL checkbox	Control or display for this ECU
Door_FR checkbox	Control or display for this ECU
Door_RL checkbox	Control or display for this ECU
Door_RR checkbox	Control or display for this ECU
Seatbelt checkbox	Control or display for this ECU
TurnSignalMode dropdown	Control or display for this ECU
BodyWarning indicator	Control or display for this ECU
BodyInputError indicator	Control or display for this ECU

Minimum CAPL test cases

CAPL test case	Minimum expected result
TC_ECU1_001_DoorOpenWhileDriving	PASS/FAIL result must be reported in CANoe test execution
TC_ECU1_002_SeatbeltWarning	PASS/FAIL result must be reported in CANoe test execution
TC_ECU1_003_TurnSignalBlink	PASS/FAIL result must be reported in CANoe test execution
TC_ECU1_004_InputTimeout	PASS/FAIL result must be reported in CANoe test execution
TC_ECU1_005_ChecksumAliveCounter	PASS/FAIL result must be reported in CANoe test execution

Required deliverable folder

```

ecu1_body_node/
src/
inc/
cmm/
  init_debug.cmm
  test_debug.cmm
canoe/
  panel file

```

CAPL test file
docs/
design.md
test_report.md

6. ECU2 - Powertrain Node Requirements

ECU2 simulates a powertrain node. It processes RPM, throttle, and engine temperature, calculates vehicle speed, detects overheating, and enters limp mode when a critical fault occurs.

Input requirements

Input	Source	Range / Encoding	Notes
EngineRPM	Arduino or CANoe	0..8000 rpm	Out-of-range RPM must be rejected.
ThrottlePercent	Arduino or CANoe	0..100 %	Used for optional acceleration behavior.
EngineTemp	Arduino or CANoe	-40..150 degC	Warning > 105 degC, critical > 120 degC.
Reset/Fault commands	CANoe	Event-based	Used for fault injection and recovery.

Output requirements

Output	Message / Interface	Cycle / Trigger	Content
0x201	PowertrainStatus	50 ms	VehicleSpeed, EngineRPM, EngineTemp, ThrottlePercent, EngineWarning, EngineCriticalFault, LimpModeActive, InputError, AliveCounter, Checksum
UART	Debug log	Event/periodic	RPM/speed/temp update, limp mode, overheat, input error
Optional GPIO LEDs	Fault indicators	Event	Engine warning and critical fault indicators

Detailed requirements

Requirement ID	Priority	Requirement Statement	Acceptance Criteria	Evidence / Deliverable
REQ-ECU2-001	P0	ECU2 MUST receive or read EngineRPM, ThrottlePercent, and EngineTemp.	All values can be changed from CANoe or Arduino and observed in UART/CAN output.	CANoe panel and trace.
REQ-ECU2-002	P0	ECU2 MUST validate sensor ranges before using values.	RPM > 8000, throttle > 100, or temp outside -40..150 sets InputError and does not update calculations.	Invalid sensor CAPL test.
REQ-ECU2-003	P0	ECU2 MUST calculate VehicleSpeed from RPM or accepted input using a documented formula.	Example: RPM 2000 produces approximately 50 km/h if VehicleSpeed = RPM / 40. Speed is limited to 220 km/h.	Code and CANoe graph.
REQ-ECU2-004	P1	ECU2 SHOULD smooth speed changes instead of jumping instantly to target speed.	Speed gradually increases/decreases based on acceleration/deceleration step.	CANoe graph.
REQ-ECU2-005	P0	ECU2 MUST send PowertrainStatus on CAN ID 0x201 every 50 ms.	CANoe decodes all required signals with stable cycle time.	CANoe trace.
REQ-ECU2-006	P0	ECU2 MUST set EngineWarning when EngineTemp > 105 degC.	EngineTemp 110 degC results in EngineWarning = 1.	CAPL overheat test.
REQ-ECU2-007	P0	ECU2 MUST set EngineCriticalFault when EngineTemp > 120 degC.	EngineTemp 125 degC results in EngineCriticalFault = 1.	CAPL critical fault test.
REQ-ECU2-008	P0	ECU2 MUST enter limp mode when EngineCriticalFault is active.	VehicleSpeed is limited to <= 30 km/h while critical fault is active.	Limp mode test.
REQ-ECU2-009	P0	ECU2 MUST detect sensor timeout after 500 ms without valid input	InputError is set and VehicleSpeed ramps down to	Timeout test.

Requirement ID	Priority	Requirement Statement	Acceptance Criteria	Evidence / Deliverable
REQ-ECU2-010	P1	update. ECU2 SHOULD provide a recovery rule for overheat and sensor timeout.	0 or safe value. Fault clears only after temperature/input returns valid for a defined number of cycles.	Recovery test.

Required CANoe panel elements

CANoe panel element	Purpose
RPM slider	Control or display for this ECU
Throttle slider	Control or display for this ECU
EngineTemp slider	Control or display for this ECU
VehicleSpeed display	Control or display for this ECU
EngineWarning indicator	Control or display for this ECU
CriticalFault indicator	Control or display for this ECU
LimpMode indicator	Control or display for this ECU
InjectInvalidRPM button	Control or display for this ECU
SensorTimeout button	Control or display for this ECU

Minimum CAPL test cases

CAPL test case	Minimum expected result
TC_ECU2_001_NormalSpeed	PASS/FAIL result must be reported in CANoe test execution
TC_ECU2_002_OverheatWarning	PASS/FAIL result must be reported in CANoe test execution
TC_ECU2_003_CriticalFault	PASS/FAIL result must be reported in CANoe test execution
TC_ECU2_004_LimpMode	PASS/FAIL result must be reported in CANoe test execution
TC_ECU2_005_InvalidSensor	PASS/FAIL result must be reported in CANoe test execution
TC_ECU2_006_SensorTimeout	PASS/FAIL result must be reported in CANoe test execution

Required deliverable folder

```

ecu2_powertrain_node/
src/
inc/
cmm/
  init_debug.cmm
  test_debug.cmm
canoe/
  panel file
  CAPL test file
docs/
  design.md
  test_report.md

```

7. ECU3 - Sensor Fusion Node Requirements

ECU3 collects and validates simulated environmental and chassis sensor data. It must demonstrate I2C/SPI communication, filtering, range checking, timeout monitoring, and stuck-sensor detection.

Input requirements

Input	Source	Range / Encoding	Notes
AmbientTemp	Arduino or CANoe	-40..85 degC	Used for environment status.
LightLevel	Arduino or CANoe	0..100 %	Night mode request when < 20%.
BatteryVoltage	Arduino or CANoe	6..18 V	Low < 11 V, over-voltage > 15 V.
SteeringAngle	Arduino or CANoe	-540..+540 deg	Signed value must be handled correctly.

Output requirements

Output	Message / Interface	Cycle / Trigger	Content
0x301	SensorStatus	100 ms	AmbientTemp, LightLevel, BatteryVoltage, SteeringAngle, BatteryLow, BatteryOverVoltage, NightModeRequest, SensorError, SensorStuck, AliveCounter, Checksum
UART	Debug log	Event/periodic	Raw/filtered sensor values, validation result, timeout/stuck detection
Optional LEDs	Sensor/fault indicators	Event	Low battery, night mode, sensor error

Detailed requirements

Requirement ID	Priority	Requirement Statement	Acceptance Criteria	Evidence / Deliverable
REQ-ECU3-001	P0	ECU3 MUST receive or read ambient temperature, light level, battery voltage, and steering angle.	Each value can be controlled from Arduino/CANoe and is visible in SensorStatus.	CANoe trace.
REQ-ECU3-002	P0	ECU3 MUST validate range for all sensor inputs.	Out-of-range values set SensorError and are not used as valid output.	Invalid range CAPL test.
REQ-ECU3-003	P0	ECU3 MUST apply a moving average filter to at least one sensor.	Filtered value changes more smoothly than raw value; raw and filtered values are loggable.	UART log and code review.
REQ-ECU3-004	P0	ECU3 MUST send SensorStatus on CAN ID 0x301 every 100 ms.	CANoe decodes signed steering angle and all fault signals correctly.	DBC and CANoe trace.
REQ-ECU3-005	P0	ECU3 MUST set NightModeRequest when LightLevel < 20 %.	LightLevel 10 sets NightModeRequest = 1; LightLevel 50 clears it.	CAPL night mode test.
REQ-ECU3-006	P0	ECU3 MUST set BatteryLow when BatteryVoltage < 11 V.	Battery 10.5 V sets BatteryLow = 1.	Battery low test.
REQ-ECU3-007	P0	ECU3 MUST set BatteryOverVoltage when BatteryVoltage > 15 V.	Battery 16 V sets BatteryOverVoltage = 1.	Over-voltage test.
REQ-ECU3-008	P0	ECU3 MUST detect sensor timeout when no update is received for more than 500 ms.	Timeout sets SensorError and logs the source that timed out.	Timeout test.
REQ-ECU3-009	P1	ECU3 SHOULD detect a stuck sensor when a value remains unchanged for a configured number of samples while source is active.	FreezeSensor test sets SensorStuck = 1.	Stuck sensor test.
REQ-ECU3-010	P1	ECU3 SHOULD document I2C/SPI frame format used with Arduino.	Mentor can reproduce sensor emulator communication.	Interface document.

Required CANoe panel elements

CANoe panel element	Purpose
AmbientTemp slider	Control or display for this ECU
LightLevel slider	Control or display for this ECU
BatteryVoltage slider	Control or display for this ECU
SteeringAngle slider	Control or display for this ECU
FreezeSensor button	Control or display for this ECU
InvalidBattery button	Control or display for this ECU
SensorError indicator	Control or display for this ECU
SensorStuck indicator	Control or display for this ECU
NightModeRequest indicator	Control or display for this ECU

Minimum CAPL test cases

CAPL test case	Minimum expected result
TC ECU3_001 NormalSensor	PASS/FAIL result must be reported in CANoe test execution
TC ECU3_002 LowBattery	PASS/FAIL result must be reported in CANoe test execution
TC ECU3_003 OverVoltage	PASS/FAIL result must be reported in CANoe test execution
TC ECU3_004 NightMode	PASS/FAIL result must be reported in CANoe test execution
TC ECU3_005 SensorStuck	PASS/FAIL result must be reported in CANoe test execution
TC ECU3_006 SensorTimeout	PASS/FAIL result must be reported in CANoe test execution

Required deliverable folder

```

ecu3_sensor_fusion_node/
src/
inc/
cmm/
  init_debug.cmm
  test_debug.cmm
canoe/
  panel file
  CAPL test file
docs/
  design.md
  test_report.md

```

8. ECU4 - Cluster Display Node Requirements

ECU4 acts as the vehicle cluster. It receives status messages from other ECUs, displays speed/RPM/warnings, manages cluster states, and enters fail-safe mode when critical CAN messages are missing.

Input requirements

Input	Source	Range / Encoding	Notes
BodyStatus 0x101	ECU1	Door, seatbelt, body warning	Timeout > 300 ms.
PowertrainStatus 0x201	ECU2	Speed, RPM, temp, engine faults	Timeout > 200 ms.
SensorStatus 0x301	ECU3	Battery, night mode, sensor fault	Timeout > 300 ms.
DiagnosticStatus 0x501	ECU5	Diagnostic active/session/DTC count	Used for DIAG_MODE.
GatewayStatus 0x601	ECU6	OTA/security/gateway warning	Used for OTA_MODE or warning display.

Output requirements

Output	Message / Interface	Cycle / Trigger	Content
0x401	ClusterStatus	100 ms	ClusterState, DisplayedSpeed, DisplayedRPM, warning lamps, DiagModeLamp, OTAModeLamp, FailSafeLamp, AliveCounter, Checksum
UART	Cluster display log	500 ms or 1 s	Readable display of speed, RPM, state, warnings
GPIO LEDs	Warning lamps	Event/periodic	Door, seatbelt, engine, battery, sensor, diag, OTA, fail-safe

Detailed requirements

Requirement ID	Priority	Requirement Statement	Acceptance Criteria	Evidence / Deliverable
REQ-ECU4-001	P0	ECU4 MUST receive BodyStatus, PowertrainStatus, SensorStatus,	Each message updates the related cluster display data.	CANoe trace and UART display.

MPC5748G Automotive Cluster & Gateway Training - Requirement Specification

Requirement ID	Priority	Requirement Statement	Acceptance Criteria	Evidence / Deliverable
		DiagnosticStatus, and GatewayStatus.		
REQ-ECU4-002	P0	ECU4 MUST implement a cluster state machine with INIT, NORMAL, WARNING, DIAG_MODE, OTA_MODE, and FAIL_SAFE states.	Mentor can trigger each state from CANoe or other ECUs; state is logged and sent in ClusterStatus.	State machine document and demo.
REQ-ECU4-003	P0	ECU4 MUST send ClusterStatus on CAN ID 0x401 every 100 ms.	CANoe decodes cluster state, displayed values, lamps, alive counter, and checksum.	CANoe trace.
REQ-ECU4-004	P0	ECU4 MUST display VehicleSpeed and EngineRPM from ECU2.	Speed/RPM in UART or LEDs matches PowertrainStatus within one update period.	CANoe/UART evidence.
REQ-ECU4-005	P0	ECU4 MUST turn on DoorWarningLamp when body warning indicates door open while driving.	Door open at speed results in DoorWarningLamp = 1.	Integration test.
REQ-ECU4-006	P0	ECU4 MUST turn on EngineWarningLamp for ECU2 EngineWarning or CriticalFault.	EngineTemp fault from ECU2 appears on cluster.	Engine warning test.
REQ-ECU4-007	P0	ECU4 MUST turn on BatteryWarningLamp for low battery or over-voltage from ECU3.	Battery 10.5 V or 16 V results in lamp ON.	Battery warning test.
REQ-ECU4-008	P0	ECU4 MUST enter FAIL_SAFE if PowertrainStatus timeout exceeds 200 ms.	When ECU2 stops sending, DisplayedSpeed becomes 0 and FailSafeLamp = 1.	Timeout test.
REQ-ECU4-009	P0	ECU4 MUST enter DIAG_MODE when DiagnosticStatus indicates active diagnostic operation.	UDS session/test from ECU5 triggers DiagModeLamp.	Diagnostic integration test.
REQ-ECU4-010	P0	ECU4 MUST enter OTA_MODE when GatewayStatus indicates active OTA operation.	OTA running on ECU6 triggers OTAModeLamp and OTA_MODE.	OTA integration test.
REQ-ECU4-011	P1	ECU4 SHOULD prioritize displays when multiple warnings are active.	Critical/fail-safe warnings have higher priority than normal warning display.	Priority logic document and test.
REQ-ECU4-012	P1	ECU4 SHOULD implement readable UART display updated every 500 ms or 1 s.	UART output is human-readable and not spammed.	UART screenshot.

Required CANoe panel elements

CANoe panel element	Purpose
Speed display	Control or display for this ECU
RPM display	Control or display for this ECU
ClusterState display	Control or display for this ECU
DoorWarning lamp	Control or display for this ECU
SeatbeltWarning lamp	Control or display for this ECU
EngineWarning lamp	Control or display for this ECU
BatteryWarning lamp	Control or display for this ECU
SensorWarning lamp	Control or display for this ECU
DiagMode lamp	Control or display for this ECU
OTAMode lamp	Control or display for this ECU
FailSafe lamp	Control or display for this ECU

Minimum CAPL test cases

CAPL test case	Minimum expected result
TC ECU4_001 NormalDisplay	PASS/FAIL result must be reported in CANoe test execution
TC ECU4_002 DoorWarning	PASS/FAIL result must be reported in CANoe test execution
TC ECU4_003 EngineWarning	PASS/FAIL result must be reported in CANoe test execution
TC ECU4_004 BatteryWarning	PASS/FAIL result must be reported in CANoe test execution
TC ECU4_005 DiagnosticMode	PASS/FAIL result must be reported in CANoe test execution
TC ECU4_006 OTAMode	PASS/FAIL result must be reported in CANoe test execution
TC ECU4_007 PowertrainTimeout	PASS/FAIL result must be reported in CANoe test execution

Required deliverable folder

```
ecu4_cluster_node/
src/
inc/
cmm/
  init_debug.cmm
  test_debug.cmm
canoe/
  panel file
  CAPL test file
docs/
  design.md
  test_report.md
```

9. ECU5 - Diagnostic and UDS Node Requirements

ECU5 implements a basic diagnostic ECU. It must parse UDS requests from CANoe, handle sessions, DIDs, DTCs, clear diagnostics, ECU reset, security access, positive responses, and negative responses.

Input requirements

Input	Source	Range / Encoding	Notes
UDS Physical Request	CANoe ID 0x7E0	Event-based	Primary diagnostic request ID.
UDS Functional Request	CANoe ID 0x7DF	Event-based	Functional diagnostic request if supported.
Fault signals	ECU1/ECU2/ECU3/ECU6	Event/periodic	Used to create DTC status.
CANoe diagnostic panel	Tester input	Event-based	Manual diagnostic operations.

Output requirements

Output	Message / Interface	Cycle / Trigger	Content
UDS Response	CAN ID 0x7E8	Event	Positive and negative diagnostic responses
0x501	DiagnosticStatus	100 ms	CurrentSession, SecurityUnlocked, ActiveDTCCount, LastServiceID, LastNRC, DiagActive, AliveCounter, Checksum
UART	Diagnostic log	Event	Received service, DID, DTC status, response code

Detailed requirements

Requirement ID	Priority	Requirement Statement	Acceptance Criteria	Evidence / Deliverable
REQ-ECU5-001	P0	ECU5 MUST receive UDS physical requests on CAN ID 0x7E0 and respond on 0x7E8.	CANoe sends request and receives valid response frame.	CANoe diagnostic trace.
REQ-ECU5-002	P0	ECU5 MUST support Diagnostic Session Control service 0x10 for Default Session 0x01 and Extended Session 0x03.	0x10 0x01 returns 0x50 0x01; 0x10 0x03 returns 0x50 0x03; unsupported sub-function returns NRC 0x12.	CAPL session tests.
REQ-ECU5-003	P0	ECU5 MUST support ECU Reset service 0x11 with hard reset sub-function 0x01 or a documented reset simulation.	Positive response is sent before reset/re-initialization; session returns to default after reset.	ECU reset test.
REQ-ECU5-004	P0	ECU5 MUST support Read Data By Identifier service 0x22.	CANoe can read supported DIDs and receives NRC 0x31 for unsupported DID.	ReadDID tests.
REQ-ECU5-005	P0	ECU5 MUST provide DID 0xF187 Software Version and DID 0xF188 Hardware Version.	DID values are returned consistently and documented.	CANoe test and DID table.

MPC5748G Automotive Cluster & Gateway Training - Requirement Specification

Requirement ID	Priority	Requirement Statement	Acceptance Criteria	Evidence / Deliverable
REQ-ECU5-006	P1	ECU5 SHOULD provide DID 0xF190 VIN.	If ISO-TP is not implemented, VIN may be shortened or documented as a simplified demo DID.	CANoe test.
REQ-ECU5-007	P0	ECU5 MUST support Write Data By Identifier service 0x2E for at least one demo configuration DID.	Write is rejected in Default Session and accepted in Extended Session.	WriteDID CAPL test.
REQ-ECU5-008	P0	ECU5 MUST support Read DTC Information service 0x19 0x02.	Active DTCs can be read after a fault is injected.	DTC test.
REQ-ECU5-009	P0	ECU5 MUST support Clear Diagnostic Information service 0x14 for all DTCs.	After clearing, ReadDTC reports no active DTC or zero count.	ClearDTC test.
REQ-ECU5-010	P0	ECU5 MUST implement negative responses for unsupported service, unsupported sub-function, request out of range, and conditions not correct.	CANoe tests receive correct 0x7F responses.	Negative response tests.
REQ-ECU5-011	P1	ECU5 SHOULD implement Security Access service 0x27 using a simple seed/key algorithm.	Correct key unlocks security; wrong key returns negative response.	Security access test.
REQ-ECU5-012	P0	ECU5 MUST send DiagnosticStatus on CAN ID 0x501 every 100 ms.	Cluster can detect diagnostic mode and CANoe displays current session/DTC count.	CANoe trace.
REQ-ECU5-013	P1	ECU5 SHOULD store DTCs based on fault status from ECU2, ECU3, or ECU6.	Engine overheat creates PowertrainOverheat DTC; sensor invalid creates SensorInvalid DTC.	Integration test.
REQ-ECU5-014	P1	ECU5 SHOULD include uds_debug.cmm for Trace32.	Script sets breakpoints in parser and watches session/security/DTC variables.	CMM file and screenshot.

Required CANoe panel elements

CANoe panel element	Purpose
Default Session button	Control or display for this ECU
Extended Session button	Control or display for this ECU
ECU Reset button	Control or display for this ECU
Read SW Version button	Control or display for this ECU
Read HW Version button	Control or display for this ECU
Read VIN button	Control or display for this ECU
Read DTC button	Control or display for this ECU
Clear DTC button	Control or display for this ECU
Request Seed button	Control or display for this ECU
Send Key button	Control or display for this ECU
Last Response display	Control or display for this ECU
Current Session display	Control or display for this ECU
SecurityUnlocked indicator	Control or display for this ECU

Minimum CAPL test cases

CAPL test case	Minimum expected result
TC ECU5_001 DefaultSession	PASS/FAIL result must be reported in CANoe test execution
TC ECU5_002 ExtendedSession	PASS/FAIL result must be reported in CANoe test execution
TC ECU5_003 ReadDID	PASS/FAIL result must be reported in CANoe test execution
TC ECU5_004 UnsupportedDID	PASS/FAIL result must be reported in CANoe test execution
TC ECU5_005 WriteDID DefaultReject	PASS/FAIL result must be reported in CANoe test execution
TC ECU5_006 WriteDID ExtendedOK	PASS/FAIL result must be reported in CANoe test execution
TC ECU5_007 ReadDTC	PASS/FAIL result must be reported in CANoe test execution
TC ECU5_008 ClearDTC	PASS/FAIL result must be reported in CANoe test execution
TC ECU5_009 SecurityAccessOK	PASS/FAIL result must be reported in CANoe test execution
TC ECU5_010 SecurityAccessFail	PASS/FAIL result must be reported in CANoe test execution

Required deliverable folder

```
ecu5_diagnostic_node/
src/
inc/
cmm/
  init_debug.cmm
  test_debug.cmm
canoe/
  panel file
  CAPL test file
docs/
  design.md
  test_report.md
```

10. ECU6 - Gateway, OTA, and Security Node Requirements

ECU6 is the most advanced ECU. It acts as a gateway monitor/filter, validates alive counters and checksums, supports OTA update simulation, and implements a software-based security/HSM mock for OTA authorization and message authentication.

Input requirements

Input	Source	Range / Encoding	Notes
CAN messages from all ECUs	CAN bus	Periodic/event	Gateway must monitor ID, alive counter, checksum, and routing rules.
OTA Request 0x650	CANoe	Event-based	Start, data, end, activate, abort commands.
Security request	CANoe	Event-based	Seed/key unlock sequence before OTA.
Invalid message injection	CANoe	Event-based	Used to test filtering and security error handling.

Output requirements

Output	Message / Interface	Cycle / Trigger	Content
0x601	GatewayStatus	100 ms	GatewayState, OTASState, SecurityUnlocked, SecurityLocked, AliveError, ChecksumError, BlockedMsgCnt, SecurityErrorCounter, ActiveSWVersion, AliveCounter, Checksum
0x651	OTA_Response	Event	OTA ACK/NACK/verify result/activate response
UART	Gateway/OTA/security log	Event/periodic	Routing decision, OTA state, CRC, security result, blocked messages

Detailed requirements

Requirement ID	Priority	Requirement Statement	Acceptance Criteria	Evidence / Deliverable
REQ-ECU6-001	P0	ECU6 MUST monitor or route allowed CAN IDs 0x101, 0x201, 0x301, 0x401, and 0x501.	Valid messages are accepted and reflected in gateway status/diagnostics.	CANoe trace and source code.
REQ-ECU6-002	P0	ECU6 MUST block or count unknown CAN IDs.	Injecting an unknown CAN ID increments GatewayBlockedMsgCnt.	CAPL invalid ID test.
REQ-ECU6-003	P0	ECU6 MUST validate checksum for monitored primary messages.	Wrong checksum sets GatewayChecksumError and increments blocked/error counter.	Checksum error test.

MPC5748G Automotive Cluster & Gateway Training - Requirement Specification

Requirement ID	Priority	Requirement Statement	Acceptance Criteria	Evidence / Deliverable
REQ-ECU6-004	P0	ECU6 MUST monitor alive counters for selected ECUs.	If alive counter does not change for N cycles, GatewayAliveError is set.	Alive counter freeze test.
REQ-ECU6-005	P0	ECU6 MUST implement OTA states: IDLE, START, RECEIVING, VERIFYING, READY_TO_ACTIVATE, SUCCESS, FAILED.	CANoe can trigger and observe each state through GatewayStatus.	OTA state-machine test.
REQ-ECU6-006	P0	ECU6 MUST support OTA_REQUEST CAN ID 0x650 and OTA_RESPONSE CAN ID 0x651.	CANoe sends OTA command and receives ACK/NACK response.	CANoe OTA trace.
REQ-ECU6-007	P0	ECU6 MUST support OTA commands START, DATA, END, ACTIVATE, and ABORT.	Valid command sequence completes successfully; invalid sequence returns negative response.	OTA protocol tests.
REQ-ECU6-008	P0	ECU6 MUST store OTA data in a bounded buffer of at least 256 bytes.	Normal 100-byte update succeeds; overflow attempt is rejected safely.	OTA overflow test.
REQ-ECU6-009	P0	ECU6 MUST verify OTA data using CRC/checksum before activation.	Correct CRC results in VERIFY_OK; wrong CRC results in OTA_FAILED.	OTA CRC tests.
REQ-ECU6-010	P0	ECU6 MUST require security unlock before accepting OTA_START.	OTA_START before unlock is rejected; OTA_START after correct key is accepted.	Security/OTA CAPL test.
REQ-ECU6-011	P0	ECU6 MUST implement seed/key security access for OTA.	Correct key unlocks; wrong key increments SecurityErrorCounter.	Security access test.
REQ-ECU6-012	P1	ECU6 SHOULD implement OTA_DATA authentication byte.	Data chunk with wrong AuthByte is rejected.	Auth test.
REQ-ECU6-013	P1	ECU6 SHOULD lock OTA after three failed security attempts.	After three wrong keys/auth failures, SecurityLocked = 1 and OTA is rejected until reset or timeout.	Security lock test.
REQ-ECU6-014	P0	ECU6 MUST send GatewayStatus on CAN ID 0x601 every 100 ms.	CANoe and ECU4 can decode gateway, OTA, and security status.	CANoe trace.
REQ-ECU6-015	P1	ECU6 SHOULD update ActiveSWVersion after successful OTA activation.	ActiveSWVersion changes only after verified OTA and activate command.	OTA success test.
REQ-ECU6-016	P1	ECU6 SHOULD provide gateway_debug.cmm, ota_debug.cmm, and security_debug.cmm.	Scripts set breakpoints and watch key variables for gateway, OTA, and security logic.	CMM files and screenshot.

Required CANoe panel elements

CANoe panel element	Purpose
Request Seed button	Control or display for this ECU
Send Correct Key button	Control or display for this ECU
Send Wrong Key button	Control or display for this ECU
OTA Start button	Control or display for this ECU
Send OTA Chunk button	Control or display for this ECU
OTA End Correct CRC button	Control or display for this ECU
OTA End Wrong CRC button	Control or display for this ECU
OTA Activate button	Control or display for this ECU
OTA Abort button	Control or display for this ECU
OTA State indicator	Control or display for this ECU
SecurityUnlocked indicator	Control or display for this ECU
SecurityLocked indicator	Control or display for this ECU
BlockedMessageCount display	Control or display for this ECU

CANoe panel element	Purpose
GatewayChecksumError indicator	Control or display for this ECU

Minimum CAPL test cases

CAPL test case	Minimum expected result
TC_ECU6_001_GatewayAllowValidMessage	PASS/FAIL result must be reported in CANoe test execution
TC_ECU6_002_GatewayBlockInvalidID	PASS/FAIL result must be reported in CANoe test execution
TC_ECU6_003_GatewayBlockWrongChecksum	PASS/FAIL result must be reported in CANoe test execution
TC_ECU6_004_SecurityUnlockOK	PASS/FAIL result must be reported in CANoe test execution
TC_ECU6_005_SecurityUnlockFail	PASS/FAIL result must be reported in CANoe test execution
TC_ECU6_006_OTAResjectWithoutSecurity	PASS/FAIL result must be reported in CANoe test execution
TC_ECU6_007_OTASuccess	PASS/FAIL result must be reported in CANoe test execution
TC_ECU6_008_OTACRCFail	PASS/FAIL result must be reported in CANoe test execution
TC_ECU6_009_OTAOverflow	PASS/FAIL result must be reported in CANoe test execution
TC_ECU6_010_SecurityLockAfter3Failures	PASS/FAIL result must be reported in CANoe test execution

Required deliverable folder

```

ecu6_gateway_ota_security_node/
src/
inc/
cmm/
  init_debug.cmm
  test_debug.cmm
canoe/
  panel file
  CAPL test file
docs/
  design.md
  test_report.md

```

11. Integration Scenarios

Requirement ID	Priority	Requirement Statement	Acceptance Criteria	Evidence / Deliverable
REQ-INT-001	P0	All six boards MUST run together on the same CAN network without CAN ID conflict.	CANoe sees messages 0x101, 0x201, 0x301, 0x401, 0x501, and 0x601 concurrently.	Full-system CANoe trace.
REQ-INT-002	P0	The system MUST support a Normal Driving scenario.	Speed 40 km/h, RPM 1800, temp 85 degC, doors closed, seatbelt fastened, battery 12.5 V results in Cluster NORMAL and no active warnings.	Integration test report.
REQ-INT-003	P0	The system MUST support Door Open While Driving scenario.	Speed 40 km/h + Door_FL open results in ECU1 BodyWarning and ECU4 DoorWarningLamp.	CANoe trace and panel screenshot.
REQ-INT-004	P0	The system MUST support Engine Overheat scenario.	EngineTemp 125 degC results in ECU2 CriticalFault, limp mode, ECU4 EngineWarningLamp, and ECU5 DTC storage.	CAPL integration test.
REQ-INT-005	P0	The system MUST support Sensor Fault scenario.	Battery 16 V or sensor timeout results in ECU3 SensorError/BatteryOverVoltage and ECU4 warning lamp.	Fault injection evidence.
REQ-INT-006	P0	The system MUST support Diagnostic scenario.	CANoe starts Extended Session, reads DID, reads/clears DTC, and ECU4 displays diagnostic mode.	UDS trace and CAPL result.
REQ-INT-007	P0	The system MUST support OTA Success scenario.	CANoe unlocks security, sends OTA data, verifies CRC, activates update, and ECU6 updates ActiveSWVersion.	OTA trace and GatewayStatus.
REQ-INT-008	P0	The system MUST support OTA Fail scenario.	Wrong CRC results in OTA_FAILED and ActiveSWVersion remains unchanged.	OTA negative test.
REQ-INT-009	P0	The system MUST support	Invalid ID or wrong checksum is	Gateway CAPL test.

Requirement ID	Priority	Requirement Statement	Acceptance Criteria	Evidence / Deliverable
		Gateway Security scenario.	blocked/counted by ECU6 and shown in CANoe panel.	

12. CAN Matrix

CAN ID	Sender	Receiver	Cycle	Message	Main Signals / Purpose
0x101	ECU1	All/CANoe	100 ms	BodyStatus	Door_FL/FR/RL/RR, Seatbelt, TurnSignalMode, HazardActive, BodyWarning, InputError, AliveCounter, Checksum
0x201	ECU2	All/CANoe	50 ms	PowertrainStatus	VehicleSpeed, EngineRPM, EngineTemp, ThrottlePercent, EngineWarning, CriticalFault, LimpMode, InputError, AliveCounter, Checksum
0x301	ECU3	All/CANoe	100 ms	SensorStatus	AmbientTemp, LightLevel, BatteryVoltage, SteeringAngle, BatteryLow, OverVoltage, NightModeRequest, SensorError, SensorStuck, AliveCounter, Checksum
0x401	ECU4	All/CANoe	100 ms	ClusterStatus	ClusterState, DisplayedSpeed, DisplayedRPM, warning lamps, DiagMode, OTAMode, FailSafe, AliveCounter, Checksum
0x501	ECU5	All/CANoe	100 ms	DiagnosticStatus	CurrentSession, SecurityUnlocked, ActiveDTCCount, LastServiceID, LastNRC, DiagActive, AliveCounter, Checksum
0x601	ECU6	All/CANoe	100 ms	GatewayStatus	GatewayState, OTAMode, SecurityUnlocked, SecurityLocked, AliveError, ChecksumError, BlockedMsgCnt, SecurityErrorCounter, ActiveSWVersion, AliveCounter, Checksum
0x650	CANoe	ECU6	Event	OTA_Request	Command, sequence, data bytes, CRC/AuthByte as defined by ECU6

CAN ID	Sender	Receiver	Cycle	Message	Main Signals / Purpose
0x651	ECU6	CANoe	Event	OTA_Response	ACK/NACK, verify OK/fail, activation result
0x7DF	CANoe	ECU5/ECU6	Event	UDS Functional Request	Functional diagnostic request
0x7E0	CANoe	ECU5	Event	UDS Physical Request	Physical diagnostic request
0x7E8	ECU5	CANoe	Event	UDS Response	Positive/negative diagnostic response

13. CANoe, Trace32, and Arduino Expectations

13.1 CANoe expectations

- CANoe configuration must include a DBC file with all project messages and signals.
- Each ECU must have at least one related panel section or panel page.
- Each ECU must have CAPL tests with clear PASS/FAIL reporting.
- CANoe must be able to inject normal values, boundary values, invalid values, timeouts, and gateway/security attacks.
- CANoe trace logs must be saved as evidence for final review.

13.2 Trace32 expectations

- init_debug.cmm must connect to target, reset target, load ELF, set breakpoint at main, and run to main.
- test_debug.cmm must set breakpoints for CAN Tx and CAN Rx and watch at least three ECU-specific variables.
- Advanced ECUs must provide extra scripts such as uds_debug.cmm, ota_debug.cmm, gateway_debug.cmm, or security_debug.cmm.
- CMM scripts should be repeatable by mentor without many manual steps.

13.3 Arduino expectations

- Arduino firmware must document the I2C/SPI frame format if used.
- Arduino must be able to produce normal sensor values and at least one fault condition.
- Arduino UART output should help verify sensor values and communication status.
- If Arduino is not available during a test, CANoe simulation may be used as fallback, but the interface design must still be documented.

14. Required Deliverables and Folder Structure

```

MPC5748G_Automotive_Cluster_Gateway_Project/
common/
  can/
  uart/
  timer/
  checksum/
  logger/
ecu1_body_node/
ecu2_powertrain_node/
ecu3_sensor_fusion_node/
ecu4_cluster_node/
ecu5_diagnostic_node/
ecu6_gateway_ota_security_node/
arduino/
  body_sensor_emulator/
  powertrain_sensor_emulator/
  environment_sensor_emulator/
canoe/

```

```

dbc/
panels/
capl_tests/
logs/
trace32/
common_scripts/
docs/
architecture.md
can_matrix.md
integration_test_report.md

```

Deliverable	Mandatory Content
Source code	Drivers, services, application logic, ECU-specific modules, build configuration.
Header files	Public interfaces, signal structures, configuration constants, requirement-related enums.
CAN description	CAN IDs, cycle time, signals, scaling, alive counter, checksum, sender/receiver.
State machine document	States, transitions, events, error handling, recovery conditions.
CANoe evidence	Panel screenshot, trace screenshot, CAPL test result.
Trace32 evidence	CMM scripts, breakpoint/watch screenshot, optional memory/register dump.
Test report	Test objective, setup, steps, expected result, actual result, PASS/FAIL, evidence link.
Demo evidence	Live demo or video covering individual ECU and integration scenario.

15. Evaluation Rubric and Review Checklist

Category	Points	Review Focus
Board bring-up, UART, GPIO, timer	10	Boot stability, heartbeat LED, UART logs, scheduler.
CAN communication and DBC correctness	20	Tx/Rx, cycle time, DBC decode, alive counter, checksum.
Sensor/CANoe/Arduino integration	15	Input source, range handling, fault injection, interface documentation.
Application logic and state machine	20	Correct behavior, state transitions, warning/fault logic, safe behavior.
Error handling and recovery	10	Timeout, invalid input, checksum/alive error, recovery behavior.
Trace32 usage and CMM scripts	10	Reusable CMM, breakpoints, watched variables, debug evidence.
CANoe panel and CAPL tests	10	Panel usability, automated tests, PASS/FAIL evidence.
Documentation and demo quality	5	Readable docs, traceability, final demo clarity.

15.1 Mentor review checklist

- ☐ Board boots successfully
- ☐ UART log is readable
- ☐ Heartbeat LED works
- ☐ CAN Tx works
- ☐ CAN Rx works
- ☐ DBC decode is correct
- ☐ Timeout handling exists
- ☐ Invalid input handling exists
- ☐ Application logic/state machine is clear
- ☐ Trace32 CMM scripts work
- ☐ CANoe panel exists
- ☐ CAPL tests run with PASS/FAIL result

- [] Documentation is complete
- [] ECU integrates with at least two other boards
- [] Full-system demo scenario passes

Appendix A - Suggested UDS DID and DTC Table

Identifier	Meaning	Owner	Notes
DID 0xF187	Software Version	ECU5 / ECU6	Returned by ReadDataByIdentifier.
DID 0xF188	Hardware Version	ECU5	Board or project hardware version.
DID 0xF190	VIN or demo VIN	ECU5	Can be shortened if ISO-TP is not implemented.
DID 0x1234	Current Fault Status	ECU5	Summarizes active system faults.
DID 0x1235	Last Reset Reason	ECU5	Useful for reset test.
DID 0x1236	Demo Configuration	ECU5	Writable only in Extended Session.
DTC 0x100001	Body communication timeout	ECU5	Set based on ECU1/body timeout status.
DTC 0x200001	Powertrain overheat	ECU5	Set when ECU2 critical overheat is active.
DTC 0x300001	Sensor invalid	ECU5	Set when ECU3 SensorError is active.
DTC 0x600001	Gateway security error	ECU5	Set when ECU6 reports security violation.

Appendix B - Recommended Final Demo Flow

1. Power on all six boards and verify UART boot logs.
2. Open CANoe configuration and confirm all periodic messages are visible and decoded.
3. Run Normal Driving scenario and verify Cluster NORMAL state.
4. Inject Door Open While Driving and verify BodyWarning and Cluster lamp.
5. Inject Engine Overheat and verify limp mode, Cluster warning, and DTC storage.
6. Run UDS ReadDID, ReadDTC, ClearDTC from CANoe.
7. Run OTA success sequence with security unlock and correct CRC.
8. Run OTA failure sequence with wrong CRC.
9. Inject invalid CAN ID/checksum and verify Gateway blocks or counts it.
10. Use Trace32 CMM on at least one ECU to show breakpoint/watch automation.